

**APIS E INTEGRAÇÕES – CONCEITOS, TIPOS, FERRAMENTAS,
EXEMPLOS**

Prof.º Denilce de Almeida Oliveira Veloso

Disciplina: Programação para WEB

CAIO DOMINGUES BERNARDINO R.A.: 0030482511010

SOROCABA

Agosto / 2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 Conceito e funcionamento de uma api.....	4
2.1 Endpoints e Integrações de API.....	4
3 Tipos de API.....	5
3.1 Classificação por Arquitetura.....	5
3.2 Classificação por Escopo de Uso.....	5
4 REST x GraphQL: Comparativo entre arquiteturas.....	6
5 Métodos de Requisição HTTP.....	7
6 CONCLUSÃO.....	8
7 REFERÊNCIAS.....	9

1 INTRODUÇÃO

As APIs (Interface de Programação de Aplicações) são conjuntos de serviços e funções que permitem que diferentes componentes de software se comuniquem e troquem dados de forma simplificada. Elas funcionam como um contrato de serviço entre aplicações, definindo como as solicitações e respostas devem ser estruturadas sem que o desenvolvedor precise conhecer os detalhes internos da implementação. O propósito central das APIs é simplificar o desenvolvimento, promovendo a abstração de elementos complexos de software. No cenário atual de disrupção digital, elas são fundamentais para empresas que buscam agilidade, inovação e a criação de novos modelos de negócio.

Diante desse cenário, o presente trabalho tem como objetivo apresentar, de forma didática, os principais conceitos relacionados a APIs e integrações, contemplando sua definição, funcionamento, arquiteturas mais utilizadas (REST, SOAP, RPC, WebSocket e GraphQL), os métodos HTTP empregados na comunicação, os mecanismos de autenticação e autorização (OAuth 2.0 e JSON Web Token), os principais riscos de segurança listados pela OWASP, ferramentas de apoio ao desenvolvimento e à governança de APIs, além de discutir a integração de sistemas legados com soluções modernas baseadas em API.

2 CONCEITO E FUNCIONAMENTO DE UMA API

Uma API é um conjunto de definições e protocolos que permite que dois componentes de software se comuniquem entre si. O termo Application (Aplicação) refere-se a qualquer software com uma função distinta, e a interface pode ser entendida como o contrato de serviço entre duas aplicações, especificando como devem ocorrer as solicitações e as respostas entre elas (AWS, 2026).

O funcionamento de uma API costuma ser descrito por meio do modelo cliente-servidor: o cliente é a aplicação que envia a solicitação (requisição), enquanto o servidor é a aplicação que processa essa solicitação e devolve uma resposta. Um exemplo clássico é o de um aplicativo de previsão do tempo no smartphone (cliente) que se comunica, por meio de uma API, com o banco de dados meteorológico de um instituto (servidor), recebendo dados atualizados sobre as condições climáticas (AWS, 2026).

2.1 Endpoints e Integrações de API

Os endpoints de API são os pontos finais de contato do sistema de comunicação, isto é, os URLs de servidores, serviços ou locais digitais específicos de onde as informações são enviadas e recebidas. Eles são fundamentais tanto para a segurança quanto para a performance do sistema, uma vez que endpoints mal protegidos tornam a aplicação vulnerável a ataques, e endpoints de alto tráfego podem gerar gargalos de desempenho (AWS, 2026).

Já as integrações de API correspondem aos componentes de software responsáveis por atualizar automaticamente os dados entre clientes e servidores, permitindo, por exemplo, a sincronização automática de arquivos na nuvem ou a automação de processos de negócio entre diferentes sistemas corporativos (AWS, 2026).

3 TIPOS DE API

As APIs podem ser classificadas de acordo com dois critérios principais: o estilo arquitetural utilizado na comunicação e o escopo de uso, ou seja, quem pode acessá-las.

3.1 Classificação por Arquitetura

- REST (Representational State Transfer): É o estilo arquitetural mais popular na web, projetado para comunicações stateless (sem estado) e baseadas em recursos identificados por URIs. Utiliza métodos HTTP padrão e pode ser armazenado em cache para melhorar a performance
- GraphQL: Uma linguagem de consulta e tempo de execução que permite ao cliente solicitar exatamente os dados necessários, evitando a busca excessiva (over-fetching) ou insuficiente (under-fetching). Diferente do REST, utiliza um único endpoint para todas as operações
- SOAP e RPC: Tecnologias mais antigas; o SOAP utiliza XML para troca de mensagens e é menos flexível, enquanto o RPC foca na execução de procedimentos no servidor por parte do cliente

3.2 Classificação por Escopo de Uso

- APIs privadas: de uso interno da organização, empregadas para conectar sistemas e dados dentro da própria empresa (AWS, 2026).
- APIs públicas: abertas para uso por qualquer desenvolvedor externo, podendo ou não exigir autorização ou cobrança para seu consumo (AWS, 2026).
- APIs de parceiros: acessíveis apenas por desenvolvedores externos previamente autorizados, geralmente no contexto de parcerias comerciais entre empresas (AWS, 2026).
- APIs compostas: combinam duas ou mais APIs distintas em uma única chamada, atendendo a requisitos mais complexos de negócio (AWS, 2026).

4 REST X GRAPHQL: COMPARATIVO ENTRE ARQUITETURAS

O REST é um estilo arquitetural criado no início dos anos 2000, baseado em um protocolo de comunicação stateless (sem estado), cliente-servidor e passível de cache. Nas APIs REST, diferentes endpoints executam operações do tipo CRUD (create, read, update e delete) sobre os recursos da aplicação, retornando, geralmente, todos os dados associados a um determinado recurso em formatos como JSON ou XML (CHINA, 2026).

O GraphQL, por sua vez, foi desenvolvido internamente pelo Facebook em 2012 e disponibilizado como código aberto em 2015. Trata-se de uma especificação e linguagem de consulta que permite ao cliente solicitar, em uma única requisição, exatamente os campos de dados de que necessita, evitando os problemas de sobrecarga (over-fetching) e de múltiplas requisições que costumam ocorrer em APIs REST tradicionais (CHINA, 2026).

5 MÉTODOS DE REQUISIÇÃO HTTP

As APIs REST utilizam o protocolo HTTP como base de comunicação, valendo-se de um conjunto de métodos (também chamados de verbos HTTP) para indicar a ação que o cliente deseja realizar sobre determinado recurso. Cada método possui um significado semântico específico, conforme descrito a seguir (MDN WEB DOCS, 2025).

- GET: solicita a representação de um recurso especificado; requisições que utilizam GET devem apenas recuperar dados, sem provocar alterações no servidor.
- POST: envia dados ao servidor, normalmente utilizado para criar um novo recurso ou provocar uma mudança de estado.
- PUT: substitui todas as representações atuais do recurso de destino pelo conteúdo enviado na requisição.
- PATCH: aplica atualizações parciais a um recurso, modificando apenas os campos informados.
- DELETE: remove o recurso especificado do servidor.
- HEAD: solicita uma resposta idêntica à de uma requisição GET, porém sem o corpo da resposta, sendo útil para verificar cabeçalhos ou a existência de um recurso.
- OPTIONS: descreve as opções de comunicação disponíveis para o recurso de destino, sendo amplamente utilizado em verificações de CORS (Cross-Origin Resource Sharing).

6 CONCLUSÃO

A partir do exposto, conclui-se que as APIs desempenham papel central na arquitetura de sistemas modernos, funcionando como elo de comunicação entre aplicações distintas e viabilizando desde a integração de sistemas legados até o desenvolvimento de soluções nativas em nuvem. A compreensão de suas diferentes arquiteturas — REST, SOAP, RPC, WebSocket e GraphQL — permite que profissionais de tecnologia escolham a abordagem mais adequada a cada cenário, considerando fatores como flexibilidade, desempenho, escalabilidade e complexidade da solução.

Compreender esses conceitos, portanto, é fundamental não apenas para desenvolvedores, mas para qualquer profissional envolvido em decisões estratégicas de tecnologia da informação.

7 REFERÊNCIAS

AKAMAI. Plano de ação para você se proteger contra os riscos à segurança listados como os 10 principais riscos de segurança às APIs do OWASP: como a Akamai pode ajudar você a lidar com vulnerabilidades e ameaças comuns às APIs. [S. l.]: Akamai Technologies, set. 2024. Disponível em: <[<https://www.akamai.com/site/pt/documents/white-paper/2024/owasps-top-10-api-security-risks.pdf>]>. Acesso em: 10 ago. 2026.

AWS. O que é uma API?: explicação sobre interfaces de programação de aplicações. [S. l.]: Amazon Web Services, 2026. Disponível em: <[<https://aws.amazon.com/pt/what-is/api/>]>. Acesso em: 10 ago. 2026.

CHINA, Chrystal R. API GraphQL versus REST: qual é a diferença?. IBM Think, 2026. Disponível em: <[<https://www.ibm.com/br-pt/think/topics/graphql-vs-rest-api>]>. Acesso em: 10 ago. 2026.

MDN WEB DOCS. Métodos de requisição HTTP. [S. l.]: Mozilla, 7 jul. 2025. Disponível em: <[<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods>]>. Acesso em: 10 ago. 2026.

SENSEDIA. API Design Reference Guide. [S. l.]: Sensedia, [20--?]. Disponível em: <[<https://content.sensedia.com/hubfs/API%20Quick%20Reference%20Guide.pdf>]>. Acesso em: 10 ago. 2026.

SOUZA, Ana Júlia. Autenticação e Autorização em APIs: OAuth 2.0 e JSON Web Token (JWT). Blog Gran Cursos Online, 5 jun. 2026. Disponível em: <[<https://blog.grancursosonline.com.br/autenticacao-e-autorizacao-em-apis-oauth-2-0-e-json-web-token-jwt/>]>. Acesso em: 10 ago. 2026.

WCONNECT. Integração de Sistemas Legados com APIs Modernas. Blog Wconnect, 3 ago. 2026. Disponível em: <[<https://wconnect.com.br/blog/integracao-de-sistemas-legados-com-apis-modernas>]>. Acesso em: 10 ago. 2026.